

1.4 Git and how we branch

Git Notes

This section is about becoming comfortable with Git, so below is the task I was assigned to do and other notes that I picked up from this section.

Learning objectives

What does a commit actually store?

A commit is like a save point in your project. *What I have gathered from asking Luke is:* when GitHub is attached to VS Code, these commands, such as `git commit -m "message"`, are run in the terminal and will show two different things. It will show where your errors are and flag them in red, and when you run commits, it will send that phase/version of your work to the repository in GitHub for you to look back over.

 **Note:** When using `git commit -m "message"`, make sure the message is clear, so when you look back at it, you and others know what you were doing and why.

 **Note:** Still confused about this (4 Aug, 15:47), but it seems like it will be covered more, and I will practise it more.

`fetch` vs `pull` vs `push` — what's the difference?

`push` = When we have made changes locally, we want to update our remote repository with those changes. Transferring our local changes to our remote repository is done with a `push` command

`pull` = *Definition:* Pull requests are proposals to merge code changes into a project.

When working as a team on a project, it is important that everyone stays up to date. Any time you start working on a project, you should get the most recent changes to your local copy. With Git, you can do that with `pull`.

`pull` is a combination of two different commands:

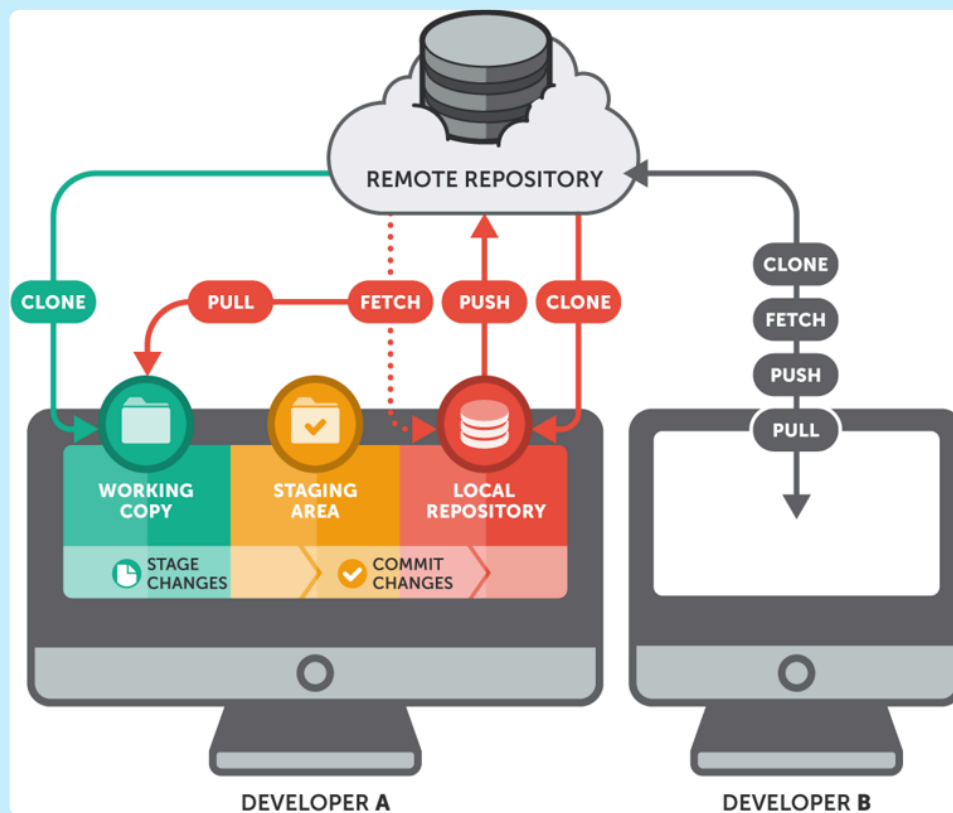
- `fetch`
- `merge`

`fetch` = *Definition:* `git fetch` downloads new data from a remote repository but does not change your working files or branches. It lets you see what others have pushed before you merge or pull.

`merge` = *Definition:* `merge` combines the current branch with a specified branch.

What I gathered from a video explaining it: You pull the local repository that you want to edit. Then, when you have edited what you want, you commit a new version of it and push it back (submit it to the repository) as a new edit or improved version. So, you merge it back with the original branch or repository.

 **LUKE EXPLANATION:** What I gathered from asking Luke: he showed me this image.



He said that when you pull from Git, you clone it, and in VS Code it is now your working copy. Then, when you click the + button on a change, it sends it to your staging area (I was not sure what loads of lines).

Note: Use `git status` again to confirm that the original/master branch is up to date.

What is a merge conflict, and how do you resolve one properly?

This is where changes are made to the same line/area of files in different branches. (I think this links into why we have a Git flow development methodology: to stop these conflicts from happening in a repository.)

Why do we branch off `main` rather than committing straight to it?

It keeps the original main project clean, keeps production code stable and clean. It stops bugs from breaking the live app. It lets many people work at the same time without crashing into each other.

What is a pull request for, beyond “getting permission to merge”?

I think it lets you discuss changes; it is a key collaboration feature in a project. (Not too sure.)

Task

✓ Create a scratch repo in the encon-group.org

First, I made a repo in GitHub—it wasn't difficult; I just navigated the website and made one.

✓ Make a branch

Then I made a new branch—it again wasn't difficult. I clicked where it said `Main` and clicked `New branch`.

Now I added the repo from GitHub to VS Code. This is cloning, and I believe it is pulling. To do this, I:

- copied the URL for my repo (located when you click the `Code` button; it was the `https://` link)
- then I opened VS Code and clicked `Command+Shift+P` to open the top bar with `>` at its start
- then I selected `Git: Clone` and pasted the URL into the bar
- on clicking Enter, I had to save the now-cloned repo to a file
- then clicked `Open`
- now the repo was opened in VS Code

Luke showed me: just open the terminal and type `git clone`, then paste the URL after it.

✓ Commit something / Push it

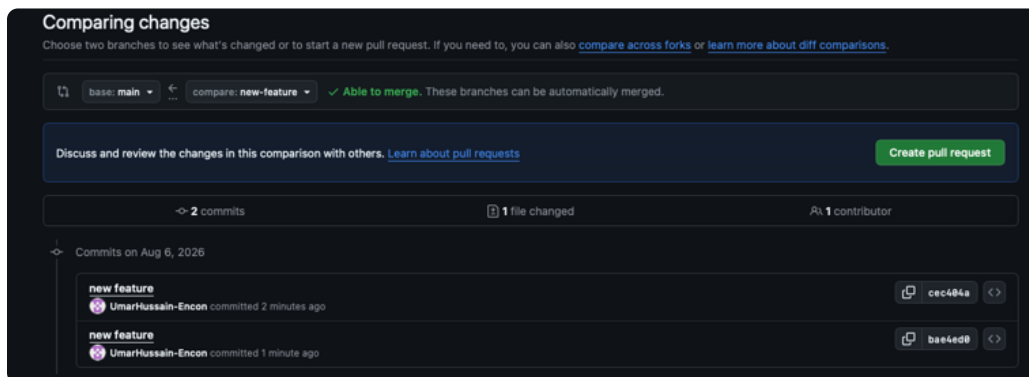
Luke showed me: then I made a change (I added a comment), and I was shown that I had to press the + button next to the change to send it to the staging area. After that, I could either click the `Commit` button or type `git commit` to “commit” the change to the GitHub repo.

✓ Open a pull request

Done in GitHub itself rather than in the VS Code terminal.

These are made when a feature wants to be merged back into the actual `main` branch.

Once an edit has been made in the new branch, and it has been staged and committed, you want to merge it with the actual `main` branch. You will submit a pull request on GitHub.



You will create a new pull request, and you want the branch you want to merge into on the left and the branch you edited on the right (as can be seen in the top left of the image above), e.g.

main ← new_feature

Then you click `Create pull request` in the top right (the big green button).



It will also show you the changes you have made compared with the branch you want to merge into (as seen in the image above).

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main ← compare: new-feature ✓ Able to merge. These branches can be automatically merged.

Add a title *

feat(SYP-454): this feature adds a new paragraph

Add a description

Write Preview

In detail analysis / description of the pul request feature

Markdown is supported Paste, drop, or click to add files

Reviewers
No reviews

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Use [Closing keywords](#) in the description to automatically close issues

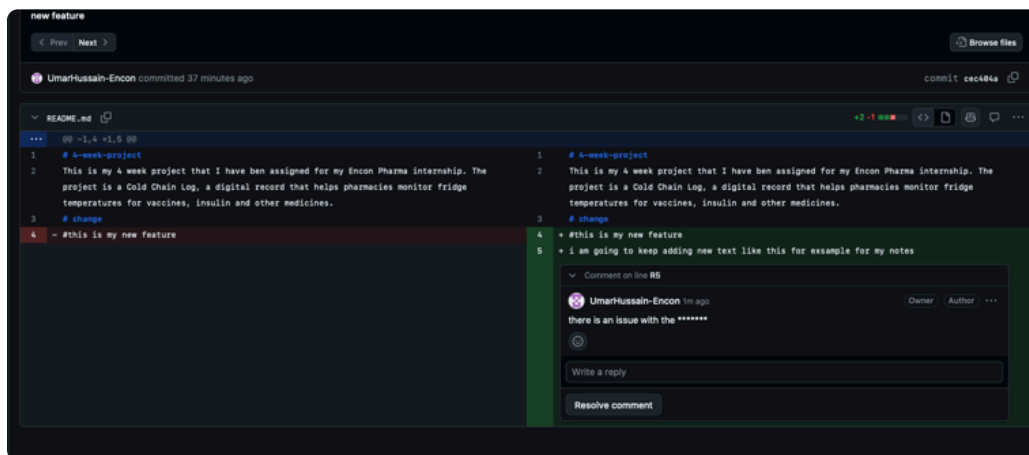
Helpful resources

Create pull request

Then you move on to writing about what the feature is. The title of the pull request will be `feat(): ; feat` stands for feature. In between the parentheses, describe the feature you are working on. Usually, the ticket number is entered there, e.g. `SYP-897`, but sometimes it can just describe the feature. There is also a description, which will go into a lot of detail about the feature.

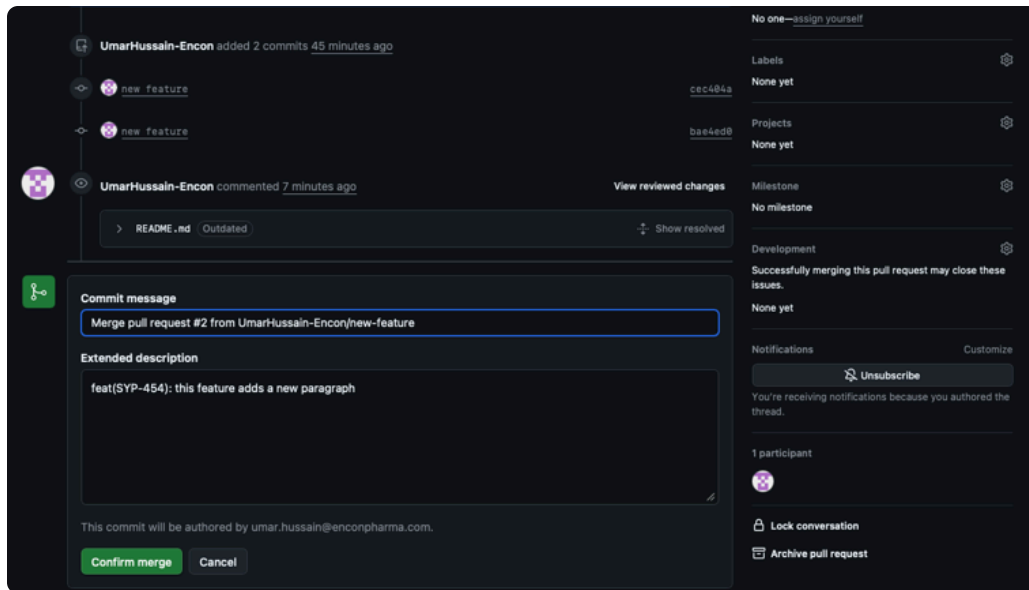
✓ Get it reviewed

Done in GitHub itself rather than in the VS Code terminal.



Now, once the pull request has been sent, it will be reviewed by other developers. They will look over the code and add comments if there are any bugs. This process goes back and forth until the code is fully ready.

✓ Merge it



Then, once it is fully corrected and confirmed to be bug-free, the feature is merged back into the `main` branch.

NOTE: `:wq` is what Luke used to exit the branch when I got stuck in the terminal.

NOTE: `git branch new-feature` used to create new branches, just have to put the name of the new branch. (can't have spaces between the names)

NOTE: When creating a new branch, `git checkout new-feature` was typed into the terminal to switch to it, and the following message was displayed to show that I was on it.

```
umarhussain@Encons-MacBook-Air 4-week-project % git branch new-feature
umarhussain@Encons-MacBook-Air 4-week-project % git checkout new-feature
Switched to branch 'new-feature'
```

When I edited the code after this point, I was editing the new branch, not the `main` branch (can you see the Git flow now?).

NOTE: `git branch -D` (Input the branch name) this command used in the terminal will delete branches.

☐ Create a merge conflict

When you merge two branches, Git tries to move the changes from one branch to the other. If the same part of the file was changed in both branches, Git may not be able to combine the changes automatically. In this case, Git will report a conflict and ask you to resolve it manually. In this lesson, we will simulate a conflict and later learn how to resolve it.

In real life merge conflicts happen regularly when working in the team. For example, you and your colleague started working on two different features, affecting the same files. Your colleague finished his work first and merged his changes to the `main` branch. Now you want to merge your own changes to the `main` branch as well. But the `main` branch is now different from the one you started working on—there is new code, submitted by your colleague. Most likely, Git will not be able to merge your changes automatically and will ask for human assistance.

24. Creating a merge conflict

When you merge two branches, Git tries to move the changes from one branch to the other. If the same part of the file was changed in both branches, Git may not be able to combine the changes automatically. In this case, Git will report a conflict and ask you to resolve it manually. In this lesson, we will simulate a conflict and later learn how to resolve it.

githowto.com

⚠ Questions, queries and pushback

After skim-reading **V2 Gitflow Workflow Encon Pharma**, I believe these have to do with Git flow, as well as coding in VS Code with GitHub. Or they both might be the same thing???

Note: The original/master is the latest saved commit and is behind by one commit???? (I believe, but I don't know; I need to ask.) (Found this when reading through W3Schools.)

- I believe this has to do with Git flow, as there are mentions of branches, `main`, and this is linked to ropes and how to work with Git in VS Code.
- **What is a merge conflict, and how do you resolve one properly?**
= I think this links into why we have a Git flow development methodology: to stop these conflicts from happening in a repository.

Need to remember to go over this stuff and re-read it; the task is important knowledge to go over.

[24. Creating a merge conflict](#) should look over this website at home, it seems like they are well explained and you can learn how

to properly git from this.